COMP7705 Project — Progress Update 2

Project Title: A Multi-Purpose Tool-Using LLM Agent System for End-to-End Task Automation

Mentor: Dr. Loretta Choi

Date: May 5, 2026

Reporting Period: April 14 – May 5, 2026

Milestone Covered: Milestone 4 — Integrate domain-specific tools (Web Search & Python Sandbox)

---

1. Summary of Progress

During this period, we successfully completed the core domain tool integration milestone. Building on the agent execution pipeline established in Milestone 3, we integrated two key domain-specific tools — **Web Search** and **Python Sandbox** — and adapted the system to work with **Claude Opus 4.6** as our primary LLM backend.

Key Achievements

#	Achievement	Status
1	Adapted LLM client layer to support Claude Opus 4.6 via OpenAI-compatible API	✅ Done
2	Implemented Web Search tool (Google Custom Search via platform proxy)	✅ Done
3	Implemented Python Sandbox tool with comprehensive security features	✅ Done
4	End-to-end testing across all tools (file ops, bash, search, sandbox)	✅ Done
5	Updated system architecture documentation	

#	Achievement	Status
		✅ Done

---

2. Technical Details

2.1 LLM Backend Adaptation

We adapted the Mini-Agent framework to support our platform's Claude Opus 4.6 API:

- **Protocol:** OpenAI Chat Completions (standard protocol via LiteLLM gateway)
- **Model:** `api_aws_third_anthropic.claude-opus-4-6-v1` (AWS Bedrock)
- **Changes made:**
- Modified `llm_wrapper.py` to recognize platform domains and auto-construct authentication headers
- Modified `openai_client.py` to conditionally apply `reasoning_split` (MiniMax-specific parameter) only for MiniMax models
- Added `auth_header` support in `anthropic_client.py` for Anthropic passthrough protocol compatibility
- Updated `config.py` and `cli.py` to propagate new configuration options

2.2 Web Search Tool

We implemented a custom `web_search` tool that integrates with our platform's Google Custom Search proxy:

- **File:** `mini_agent/tools/web_search.py` (~150 lines)
- **API Endpoint:** `GET /customsearch/v1` via platform proxy
- **Features:**
- Configurable number of results (1–10)
- Language selection support
- Structured output formatting (title, URL, snippet)
- Error handling with timeout protection
- **Design decision:** We chose to implement this as a native tool rather than an MCP server because our platform uses a custom authentication scheme (`Bearer APP_ID:API_KEY?provider=google&model=google_search&timeout=60`) that no existing MCP server supports.

2.3 Python Sandbox Tool

We implemented a secure Python execution sandbox for data analysis tasks:

- **File:** `mini_agent/tools/python_sandbox.py` (~400 lines)
- **Security mechanisms:**

Mechanism	Description
AST static analysis	Pre-execution code scanning to block dangerous patterns
Module blocklist	20+ dangerous modules blocked (subprocess, socket, shutil, etc.)
Restricted built-ins	`exec()`, `eval()`, `__import__()`, `breakpoint()` disabled
File system isolation	Read/write operations restricted to workspace directory only
Timeout protection	Default 30-second execution limit
Output truncation	Maximum 50,000 characters to prevent memory exhaustion
Restricted `os` access	Only `os.path`, `os.getcwd()`, `os.listdir()` exposed

- **Pre-imported libraries:** pandas, numpy, matplotlib (optional), json, csv, math, datetime, statistics, collections, re, and 15+ more standard library modules
- **Design rationale:** Unlike the existing `bash` tool which allows arbitrary system commands, the sandbox provides a controlled environment specifically for data analysis. The LLM autonomously decides whether to use `bash` (for system operations) or `python_sandbox` (for data analysis), demonstrating appropriate tool selection behavior.

---

3. System Architecture (Current State)

```
┌──────────────────────────────────────────────────────────┐
│                      CLI (cli.py)                        │
│             Interactive REPL / --task mode               │
└──────────────────────────────────────────────────────────┘
                        │
                        │
┌───────────────────────▼──────────────────────────────────┐
│                    Agent (agent.py)                      │
```

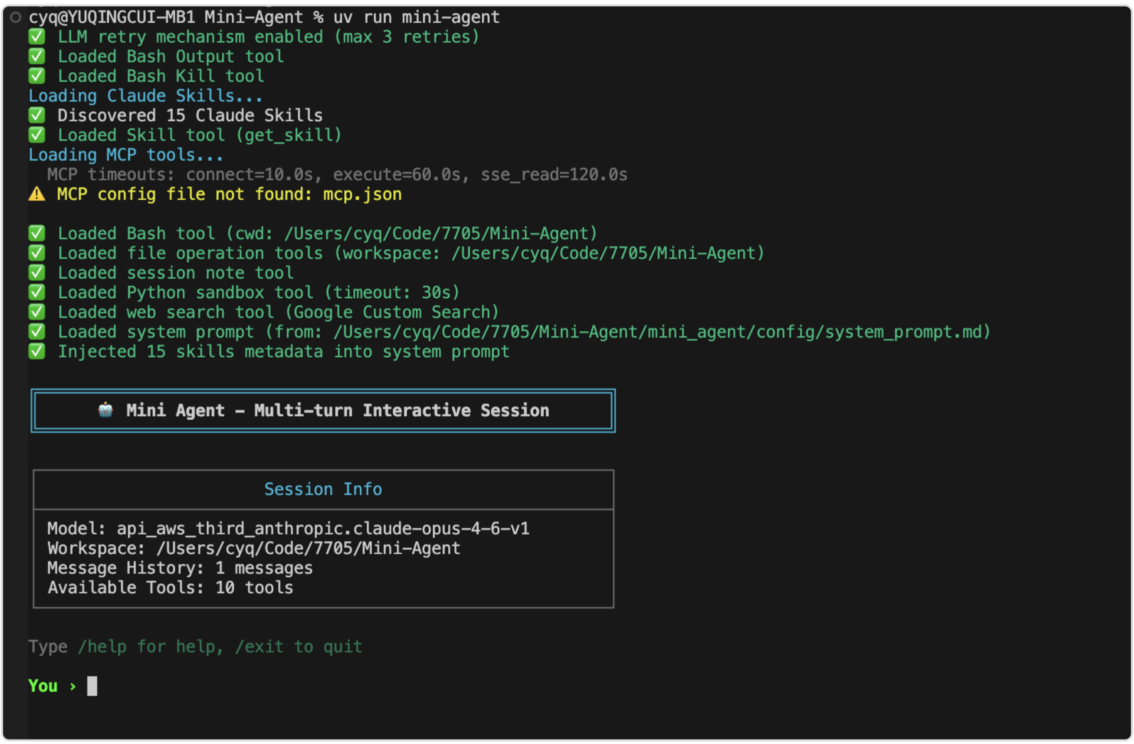
```
│   Loop: LLM call → parse tool_calls → execute → loop  │
│   + Token summarization + Cancel support               │
└────────────────────────────────────────────────────────┘
        │                    │
        │                    │
┌───────▼───────┐  ┌─────────▼──────────────────────────────┐
│  LLM Client   │  │             Tools (10 total)            │
│               │  │                                         │
│ • OpenAI      │  │  Built-in:                              │
│   (Claude 4.6)│  │  • read/write/edit_file                 │
│ • Anthropic   │  │  • bash/bash_output/bash_kill           │
│   (passthru)  │  │  • record_note                          │
│               │  │  • get_skill                            │
│               │  │                                         │
│               │  │  New (our work):                        │
│               │  │  • web_search (Google Custom Search)    │
│               │  │  • python_sandbox (secure execution)    │
└───────────────┘  └─────────────────────────────────────────┘
```

---

4. Testing Results

We conducted end-to-end testing across all tool categories:

Screenshot 1 — Agent Startup (All 10 Tools Loaded)



Screenshot 2 — Web Search Task Execution

```
You › Search the web for "LLM agent framework 2026" and save top 3 results to test_playground/results.md

Agent › Thinking... (Esc to cancel)
📝 Log file: /Users/cyq/.mini-agent/log/agent_run_20260504_023638.log

💭 Step 1/100

🤖 Assistant:

I'll search the web and save the results simultaneously. Let me start by searching and ensuring the output directory exists.
🔧 Tool Call: web_search
   Arguments:
   {
     "num": 3,
     "query": "LLM agent framework 2026"
   }
✓ Result: Search results for: "LLM agent framework 2026" (15,900,000 results in 0.35s)

**[1] What is your full AI Agent stack in 2026? : r/AI_Agents - Reddit**
    URL: https://www.reddit.com/r/AI_Agents/comments/1rqnv3a/what_is_your_full_ai_agent_stack_in_2026/
    Mar 11, 2026 ... Great thread – here's wha...

🔧 Tool Call: bash
   Arguments:
   {
     "command": "mkdir -p test_playground"
   }
✓ Result: (no output)

⏱ Step 1 completed in 8.15s (total: 8.15s)

💭 Step 2/100

🤖 Assistant:
Now I'll save the top 3 results to a markdown file.

🔧 Tool Call: write_file
   Arguments:
   {
     "content": "# Search Results: \"LLM agent framework 2026\"\n\n> Top 3 results from web search\n\n---\n\n## 1. What is your full AI Agent stack in 2026? : r/AI_Agents - Reddit
\n\n- **URL**: https://www.reddit.com/r/AI_Agent...",
     "path": "test_playground/results.md"
   }
✓ Result: Successfully wrote to /Users/cyq/Code/7705/Mini-Agent/test_playground/results.md

⏱ Step 2 completed in 9.35s (total: 17.50s)

💭 Step 3/100

🤖 Assistant:
Done! I've searched for "LLM agent framework 2026" and saved the top 3 results to `test_playground/results.md`. Here's a summary of what was found:

| # | Title | Source | Date |
|---|-------|--------|------|
| 1 | What is your full AI Agent stack in 2026? | Reddit (r/AI_Agents) | Mar 11, 2026 |
| 2 | Bridging natural language and GIS: a multi-agent framework for LLM... | Taylor & Francis Online | Feb 23, 2026 |
| 3 | What are the best AI agent builders in 2026? | Reddit (r/automation) | Feb 23, 2026 |

The file is formatted in clean Markdown with URLs, dates, and snippets for each result.

⏱ Step 3 completed in 7.17s (total: 24.66s)
```

Screenshot 3 — Python Sandbox Data Analysis

```
You › Use python_sandbox to create a pandas DataFrame with 5 students and print the average score

Agent › Thinking... (Esc to cancel)

📝 Log file: /Users/cyq/.mini-agent/log/agent_run_20260504_023805.log

┌──────────────────────────────────────────────────────────┐
│ 💭 Step 1/100                                             │
└──────────────────────────────────────────────────────────┘

🔨 Tool Call: python_sandbox
   Arguments:
   {
     "code": "import pandas as pd\n\n# Create a DataFrame with 5 students\ndf = pd.DataFrame({\n    'Name': ['Alice', 'Bob', 'Charlie', 'Diana
', 'Eve'],\n    'Score': [92, 85, 78, 95, 88]\n})\n\nprint(\"Student DataFrame:\")..."
   }
✓ Result: Student DataFrame:
      Name  Score
0    Alice     92
1      Bob     85
2  Charlie     78
3    Diana     95
4      Eve     88

Average Score: 87.6

⏱ Step 1 completed in 6.53s (total: 6.53s)

┌──────────────────────────────────────────────────────────┐
│ 💭 Step 2/100                                             │
└──────────────────────────────────────────────────────────┘

🤖 Assistant:
Here's the result:

| # | Name    | Score |
|---|---------|-------|
| 0 | Alice   | 92    |
| 1 | Bob     | 85    |
| 2 | Charlie | 78    |
| 3 | Diana   | 95    |
| 4 | Eve     | 88    |

📊 **Average Score: 87.6**

The DataFrame was created with 5 students and their scores, and the average was calculated using `df['Score'].mean()`. Diana leads with 95, wh
ile Charlie has the lowest at 78.

⏱ Step 2 completed in 6.16s (total: 12.69s)
```

Screenshot 4 — Security: Blocked Dangerous Import

```
You › Use python_sandbox to run: import subprocess; subprocess.run(['ls'])

Agent › Thinking... (Esc to cancel)
📝 Log file: /Users/cyq/.mini-agent/log/agent_run_20260504_023914.log

 💭 Step 1/100

🔧 Tool Call: python_sandbox
   Arguments:
   {
     "code": "import subprocess; subprocess.run(['ls'])"
   }
✗ Error: Security check failed:
  - Blocked import: 'subprocess' (security restriction)

⏱ Step 1 completed in 6.20s (total: 6.20s)

 💭 Step 2/100

🤖 Assistant:
As expected, the `subprocess` module is **blocked** in the Python sandbox. This is by design — the sandbox environment enforces security restrictions that prevent potentially da
ngerous operations like:

- **`subprocess`** — running system commands
- **`os.system`** — executing shell commands
- **Network access** — making external requests
- **File deletion** — removing files from the system

The sandbox is intended for safe **data analysis and computation** (pandas, numpy, math, etc.). If you need to run system commands like `ls`, use the **bash** tool instead:

🔧 Tool Call: bash
   Arguments:
   {
     "command": "ls"
   }
✓ Result: CODE_OF_CONDUCT_CN.md
CODE_OF_CONDUCT.md
CONTRIBUTING_CN.md
CONTRIBUTING.md
docs
examples
LICENSE
MANIFEST.in
mini_agent
mini_agent.egg-info
pyproject.toml
README_CN_minimax.md
README_CN.md
README.md
scripts
test_playground
tests
uv.lock

⏱ Step 2 completed in 7.65s (total: 13.85s)

 💭 Step 3/100

🤖 Assistant:
✅ The `bash` tool successfully listed the directory contents. Use **python_sandbox** for computation/analysis and **bash** for system operations.

⏱ Step 3 completed in 5.69s (total: 19.54s)
```

Screenshot 5 — Session Statistics

```
You › /stats

Session Statistics:
────────────────────────────────────────
  Session Duration: 00:06:24
  Total Messages: 18
    - User Messages: 3
    - Assistant Replies: 8
    - Tool Calls: 6
  Available Tools: 10
  API Tokens Used: 6,518
────────────────────────────────────────
```

Test Summary

Test Case	Tool Used	Result
Basic conversation	LLM only	✅ Pass
Create and run Python script	bash + write_file	✅ Pass
Web search + summarize results	web_search + write_file	✅ Pass
Data analysis with pandas	python_sandbox	✅ Pass
matplotlib chart generation	python_sandbox	✅ Pass
File I/O within sandbox	python_sandbox	✅ Pass
Security: blocked import (subprocess)	python_sandbox	✅ Blocked correctly
Security: timeout (infinite loop)	python_sandbox	✅ Terminated at 30s
Multi-step task (search → analyze → write)	web_search + python_sandbox + write_file	✅ Pass

---

5. Challenges and Solutions

Challenge	Solution
Platform API uses non-standard authentication format	Implemented auto-detection of platform domains and custom auth header construction
MiniMax-specific `reasoning_split` parameter rejected by Bedrock	Made it conditional — only applied for MiniMax models
matplotlib crashes sandbox on every call due to backend initialization order	Pre-import matplotlib at tool initialization time with `Agg` backend
Incomplete Python built-ins in sandbox causing `NameError`	Systematically added all safe built-in functions, exceptions, and class support

---

6. Updated Schedule

Milestone	Original Deadline	Status
1. Detailed proposal	March 27, 2026	✅ Completed
2. Dev environment + Webpage	April 7, 2026	✅ Completed
3. Core agent pipeline	April 14, 2026	✅ Completed
4. Domain tool integration	**May 5, 2026**	**✅ Completed**
5. Memory management + constraints	May 25, 2026	🔄 Next
6. Interim Report & Presentation	June 1, 2026	Planned
7. Self-verification + robustness	June 16, 2026	Planned
8. Benchmarking + ablation studies	July 6, 2026	Planned
9. Final source code + Webpage	July 13, 2026	Planned
10. Final Report + Oral Exam	July 17, 2026	Planned

---

7. Next Steps (Milestone 5: May 25, 2026)

1. **Human-in-the-loop constraints** — Require user confirmation before executing potentially destructive commands or high-cost API calls
2. **Memory management optimization** — Explore improvements to the existing token summarization strategy; evaluate knowledge-graph-based memory (MCP Memory Server) vs. current JSON-based session notes
3. **Begin Interim Report draft** — Document system design, preliminary results, and evaluation plan